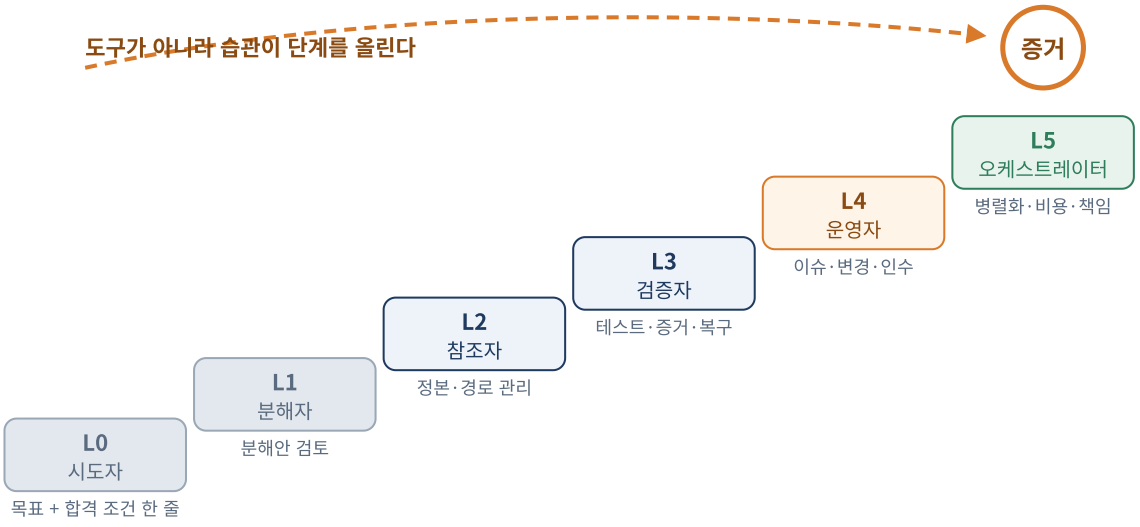
실전 성장 로드맵 · L0 시도자 → L5 오케스트레이터

AI는 "완료했습니다"라고 말합니다.
당신은 무엇을 확인하시겠습니까?

비개발자를 위한 AI 바이브코딩 성장형 가이드

결과를 정의하고, 증거로 검증하고, 위험은 기술로 제한한다

코드를 쓰지 않는 **기획자 · 현업 담당자 · 창업자 · 발주자**를 위한 습관 형성 교재 — Claude Code와 oh-my-claudecode 적용 포함



발행일 2026년 11월 10일

비개발자를 위한 AI 바이브코딩 성장형 가이드

Claude Code와 oh-my-claudecode를 포함한 실전 성장 로드맵 · 2026년 11월 10일 윤문판

> **최고 수준의 목표는 코드를 많이 만드는 사람이 아니라, 원하는 결과를 명확히 정의하고 AI가 만든 결과를 증거로 검증하며 지속적으로 개선하는 운영자가 되는 것이다.**

대상	코드를 직접 작성하지 않는 기획자·현업 담당자·창업자·발주자·AI 개발조직 운영자
다루는 환경	채팅형 AI, Claude Code, Git 기반 저장소, 이슈 관리, oh-my-claudecode(OMC)
적용 원칙	최소한의 인간 개입, 위험에 비례한 검증, 실행 가능한 결과물 중심
독서 안내	**10분 요약과 1~3장 먼저** → 작은 과제로 실습 → 해당 단계의 본문과 템플릿만 참조. 이 문서의 수치와 주기는 권장 초기값이며, 도구의 명령과 설정은 사용 직전에 최신 문서로 확인한다.

10분 요약 이 책의 사용법

처음부터 모든 장을 이해하려 할 필요가 없다. 당장 다음 **다섯 가지 습관**만 시작한다. 이 다섯 가지는 도구가 바뀌어도 남는다. `CLAUDE.md`, 참조 파일, 테스트 자동화, 다중 에이전트는 이 습관을 확장하는 수단이다.

다섯 가지 습관

1. **결과물부터 말하기**: "무엇을 만들 것인가"에 더해 "어떻게 합격을 확인할 것인가"를 한 문장으로 쓴다.
2. **AI에게 분해안을 받되 검토하기**: AI에게 3~7개 작업으로 나누게 하고 누락·순서·검증을 점검한다.
3. **한 번에 작은 결과를 확인하기**: 한 작업에서 보여 줄 결과와 합격 기준을 먼저 정한다.
4. **기억보다 정본을 남기기**: 합의한 목표·결정·진행상태·증거를 파일 또는 이슈에 남긴다.
5. **완료 선언보다 증거를 보기**: 실제 실행 화면, 테스트 결과, 외부 시스템의 합격 응답으로 판단한다.

시작할 때 쓰는 60초 지시문

```
나는 비개발자다. 최종 목표는 [결과물]이며, 사용자는 [대상]이다.
완료 여부는 [관찰 가능한 합격 조건]으로 판정한다.
먼저 해야 할 일을 3~7개로 구조화해서 보여줘.
각 항목에 ① 결과물 ② 선행조건 ③ 검증방법 ④ 실패 위험을 한 줄씩 적어줘.
내가 선택하거나 확인해야 할 사항과 네가 자율 판단해도 될 사항을 구분해.
작업을 시작하기 전 누락·중복·순서 문제를 자체 점검해줘.
```

보안·정확성·인수는 지시문이 아니라 뒤에서 다룰 실제 검증 장치가 보장한다.

1장 흔한 오해와 바로잡기

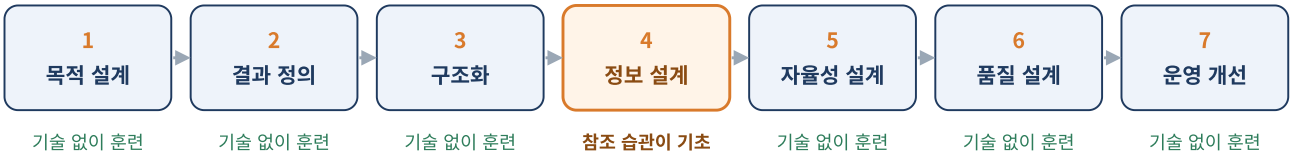
흔히 보이는 비개발자용 바이브코딩 안내는 '신입 사원에게 업무 지시하기' 비유와 "작을 때는 한 번에 시키고, 커지면 자료를 참조시켜라"는 2분법으로 시작한다. 입문에는 유용하지만 **사용 요령**과 **능력 성장·장기 운영**을 혼동하기 쉽다. AI가 잘못된 일을 얼마나 오래 실행할 수 있는지도 과소평가하기 쉽다. 다음 열두 가지가 대표적인 오해다.

흔한 오해	왜 문제인가	이렇게 한다
작을 때는 그냥 시키고, 커지면 자료를 참조시킨다	참조 습관을 너무 늦게 배운다. 작은 과제에서도 출처와 파일 위치를 익힐 수 있다.	**작게 시작하되 처음부터 출처·검증 습관**을 만든다.
헷갈리기 시작하면 그때 구조를 잡는다	문제가 터진 뒤의 대규모 구조 변경은 회귀 오류를 늘린다.	시작 시 최소 정본을 만들고, 규모가 커지면 점진적으로 분리한다.
파일 쪼개기는 AI가 알아서 한다	코드를 나누는 일과 요구사항을 나누는 일을 혼동한다. AI의 잘못된 분해도 검토가 필요하다.	**사람은 분해안의 목적·경계·누락을 검토**하고, AI는 기술적 파일 설계를 맡는다.
`CLAUDE.md`는 프로젝트의 목차다	상시 적용할 지침의 자리다. 모든 자료의 목차·일지로 만들면 비대해진다.	짧은 운영 규칙만 두고, 지식 색인·상태·이슈는 따로 둔다.
`@`로 연결하면 토큰이 절약된다	Claude Code의 `@경로` 가져오기는 내용을 함께 불러온다. 자동 비용 절감이 아니다.	**경로 안내와 자동 포함(import)**을 구분한다.[S1]
200줄 이내는 절대 규칙이다	공식 문서가 권하는 *목표치*이지 보안 규칙이 아니다.	간결하고 구체적인 규칙, 도구 버전 확인, 충돌 점검.[S1]
파일이 3~4개가 되면 방식을 바꾼다	파일 개수는 임의 지표다. 규칙의 반복, 경계 불명확, 긴 문맥, 인수 기준이 더 중요하다.	상태·위험·변경 빈도·검증 필요성에 따라 전환한다.
성공 사례의 매출 수치가 근거다	1차 자료·시점·성과 정의가 없는 사례는 롤모델의 근거가 되기 어렵다.	출처 미확인 사례와 숫자는 빼고, 재현 가능한 실습·측정에 집중한다.
보안 스캔을 돌렸으니 안전하다	스캔 한 번으로 권한·비밀정보·운영 위험을 모두 해결할 수 없다.	보안 경계 + 테스트 + 리뷰 + 배포 승인으로 다층 확인한다.[S6 S7]
오늘 정한 것을 `CLAUDE.md`에 넣는다	일회성 진행상황이 상시 지침을 오염시킨다.	결정은 `DECISIONS.md`, 진행은 이슈나 `STATUS.md`, 변하지 않는 규칙만 `CLAUDE.md`.
좋은 도구를 쓰면 수준이 오른다	제품·버전·권한·요금은 변하고, 산출물의 구조가 남는다.	도구 중립 원칙 위에 Claude Code·OMC의 구체 적용을 얹는다.
AI가 완료했다면 완료다	PR·커밋·테스트와 고객 인수는 서로 다른 사건이다.	**작동하는 제품·사용자 행동·외부 판정**으로 끝을 정의한다.

이 책은 '프롬프트 방식이냐 참조 방식이냐'의 2분법 대신 **목표 설계 → 구조화된 분해 → 관련 자료 탐색 → 실행 → 검증 → 인수 → 학습**의 **일곱 역량**을 단계적으로 쌓도록 설계했다.

2장 높은 롤모델의 정의

최고 수준의 비개발자란 어려운 기술 용어를 아는 사람이나 가장 많은 에이전트를 띄우는 사람이 아니다. 다음 일곱 가지 일을 안정적으로 해내는 사람이다.



일곱 역량 중 여섯은 코드를 몰라도 훈련할 수 있다 — 4번의 기초가 포인터·참조 습관이다

그림 1. 비개발자가 쌓는 일곱 역량

1. **목적 설계**: 사업 목적과 사용자에게 생기는 변화를 설명한다.
2. **결과 정의**: 출력 형식, 예외 상황, 실패 조건을 구체화한다.
3. **구조화**: 일을 단계·기능·정보·위험으로 나누고 관계를 설명한다.
4. **정보 설계**: AI가 필요한 근거를 어디에서 어떻게 찾아야 하는지 지정한다.
5. **자율성 설계**: 사람이 결정할 일과 AI에 맡길 일을 구분한다.
6. **품질 설계**: 실제 합격 기준과 증거를 요구한다.
7. **운영 개선**: 오류의 원인을 기록하고 다음 프로젝트에 재사용한다.

인간과 AI의 역할 경계

사람이 끝까지 책임질 것	AI에게 폭넓게 위임할 것
무엇을 왜 만들어야 하는가	대안 조사·기술 후보 탐색
사용자와 사업상 필수 기준	요구사항의 기술적 세분화 초안
개인정보·권한·예산·법적 책임의 한계	설계·코딩·테스트 생성·문서화
위험이 큰 외부 행위와 인수 결정	허용된 범위에서 반복 수정 및 품질 검사
최종 합격과 경제적 가치 판단	검증 결과 수집·요약·실패 원인 분석

최소 통제는 *위험이 낮고 검증이 가능한 작업은 넓게 위임하고, 돌이키기 어려운 행동은 기술적으로 막거나 승인받게 하는 설계*다.

3장 성장을 위한 여섯 단계

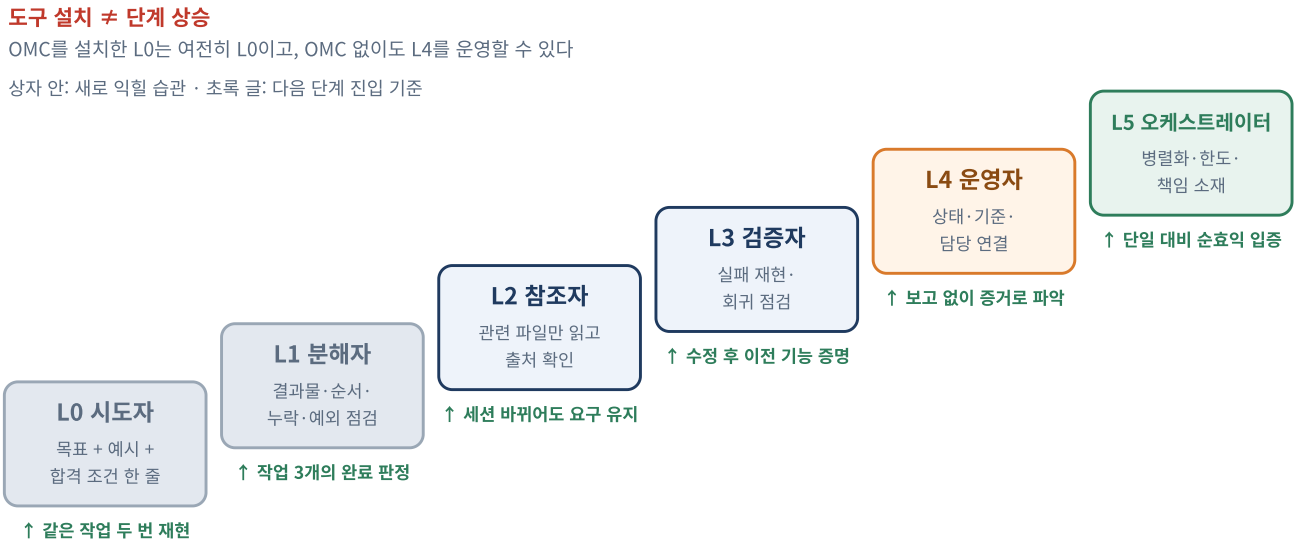


그림 2. 여섯 단계의 새 습관과 다음 단계 진입 기준

단계	상태	새로 익힐 핵심 습관	다음 단계 진입 기준
L0 시도자	채팅으로 기능을 요청한다.	목표 + 화면/파일 예시 + 합격 조건 한 줄	같은 작업을 두 번 재현할 수 있다.
L1 분해자	작업을 나누고 검토한다.	결과물·순서·누락·예외를 점검한다.	3개 이상 작업의 개별 완료를 판정한다.
L2 참조자	자료의 정본과 경로를 관리한다.	관련 파일만 읽고 출처를 확인한다.	세션이 바뀌어도 요구사항이 왜곡되지 않는다.
L3 검증자	테스트·증거·버전 복구를 운용한다.	실패를 재현하고 회귀를 점검한다.	수정 이후 이전 기능이 정상임을 증명한다.
L4 운영자	이슈·변경·인수를 관리한다.	일의 상태와 기준, 담당을 연결한다.	진행 보고 없이도 대시보드·증거로 상황을 안다.
L5 오케스트레이터	여러 AI·도구·역할을 조율한다.	독립 작업 병렬화, 비용·실패 제한, 책임소재	단일 에이전트 대비 순효익을 측정해 입증한다.

단계를 건너뛰면 안 된다는 뜻은 아니다. 도구 설치와 성장 단계를 혼동하지 않는다.

매일의 5분 루틴

시작 전 2분

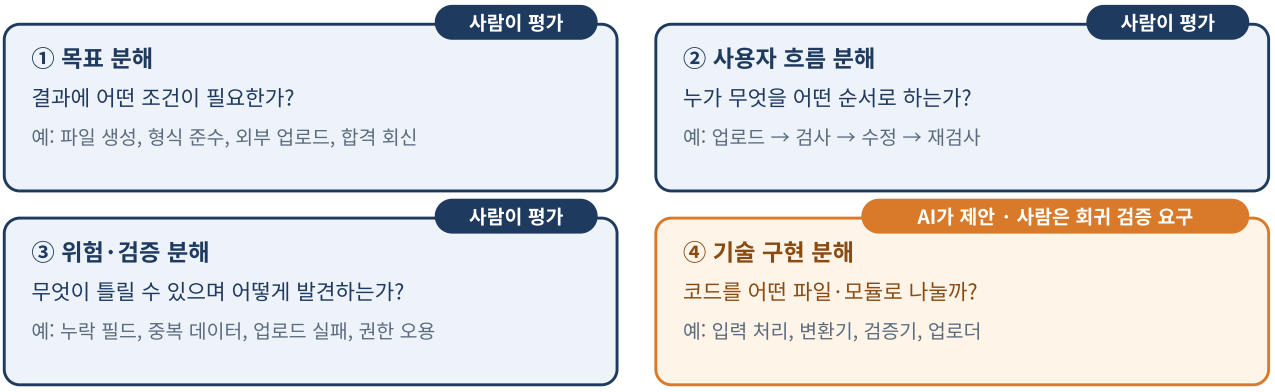
- 오늘 확인할 최종 결과는 무엇인가?
- AI에게 줄 자료 중 **정본**은 어느 것인가?
- 무엇이 나오면 **합격**인가?
- AI가 임의로 바꾸면 안 되는 대상은?

끝날 때 3분

- 결과를 실제로 열고 확인했는가?
- 통과·실패 증거는 어디에 있는가?
- 바뀐 결정과 남은 이슈는 기록했는가?
- 다시 시작할 수 있는 상태인가?

4장 가장 중요한 능력인 구조화된 쪼개기

'쪼개기'에는 **네 개의 다른 행위**가 있다. 먼저 이 넷을 구분한다. **사람은 앞의 세 가지를 평가할 능력을 기른다.** 기술 구현 분해는 AI가 제안하게 하되, 통합과 회귀 검증을 요구한다.



"한 작업 = 한 파일"은 잘못된 규칙이다 — 병렬 작업은 같은 파일을 동시에 고치지 않는 경계까지 설계한다

그림 3. 쪼개기의 네 종류 중 사람이 평가하는 셋과 AI가 제안하는 하나

파일을 만들고 외부 사이트에서 합격받는 실전 예시

막연한 지시: "11개 상용 소프트웨어의 공통 기능을 조사해 어떤 파일을 만들어 사이트에서 통과하게 해." 이를 **결과를 기준으로** 1차 분해한다.

번호	작업 — 과정이 아니라 결과로 쓴다	완료 증거	선행조건
T1	목표 파일 형식과 판정 기준 확정	명세, 정상/비정상 예시	외부 사이트 규칙 입수
T2	11개 제품 공통 기능의 비교표 완성	출처·버전·선정 근거 표시된 표	조사 범위 확정
T3	최소 입력→출력 변환기 작동	샘플 입력과 출력 파일	T1
T4	형식·필수 항목 자동 검사 통과	검사 기록과 실패 샘플	T1·T3
T5	외부 사이트 업로드 및 합격	접수 ID·응답·합격 증거	T3·T4·권한
T6	실패 재현·수정·재시험 경로 확보	실패 이슈와 재실행 기록	T5의 실패/변경

2차 분해에서는 작업 하나의 적정 크기를 정한다. 작업은 '아주 작게'가 아니라 **독립적으로 확인할 수 있을 만큼** 쪼갠다. 다음 질문 중 두 개 이상에 '아니오'면 더 분해하거나 경계를 다시 정한다.

1. 산출물 하나 또는 명확한 상태 변화로 설명되는가?
2. 시작 전 필요한 입력과 의존 작업이 정해졌는가?
3. 완료했는지 타인이 동일하게 확인할 수 있는가?
4. 실패해도 다른 작업에 미치는 영향을 찾을 수 있는가?
5. 담당/책임 소재를 명확히 지정할 수 있는가?

비개발자의 구조화 검토 6문항

1. **누락**: 사용자는 성공뿐 아니라 실패 시 무엇을 보게 되는가?
2. **중복**: 서로 다른 작업이 같은 기능을 두 번 만들지 않는가?
3. **순서**: 아직 확보하지 않은 자료에 의존하는 작업이 있는가?
4. **경계**: 한 작업을 수정하면 어떤 다른 작업이 영향을 받는가?
5. **판정**: '완성'은 보기 좋은 화면인가, 실제 작동인가, 외부 검증 통과인가?
6. **복구**: 중단되거나 실패하면 어디에서 다시 시작하는가?

분해 검토 프롬프트

```
당신은 지금 구현자가 아니라 '작업 분해안을 검토하는 기획자'다.
내 목표와 합격 기준을 먼저 5줄 이내로 재진술하라.
작업을 [사용자 흐름 / 데이터 / 외부 연동 / 검증 / 운영] 관점에서 나눠라.
작업마다 결과물, 선행작업, 검증방법, 실패 시 영향, 불명확한 가정을 적어라.
누락·중복·순환 의존성·과도한 세분화를 스스로 비판하라.
기술적 구현 파일 분해는 아직 수행하지 말라.
내가 승인해야 하는 결정만 별도 목록으로 제시하라.
```

5장 포인터·참조 방식의 정확한 이해

포인터는 어떤 자료가 어디 있고 언제 읽어야 하는지 적어 둔 지시다. **지시하는 방법에 따라 실제로 AI에 입력되는 정보량이 다르다.** 네 가지를 반드시 구분한다.

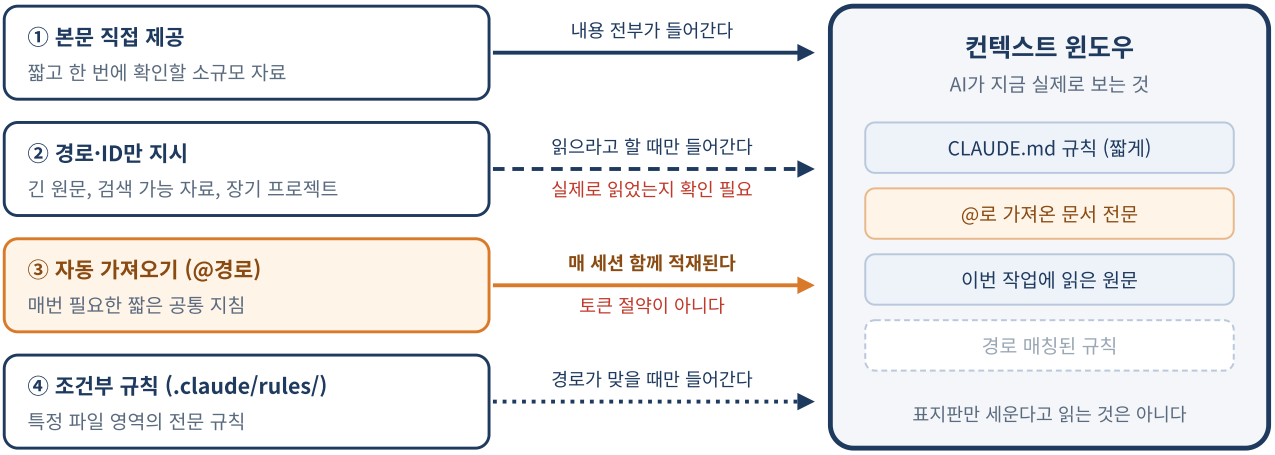


그림 4. 참조의 네 방식과 실제로 컨텍스트에 들어가는 것의 차이

방식	무슨 일이 일어나는가	적합한 용도	주의점
본문 직접 제공	중요한 내용 자체를 대화에 넣는다.	짧고 한 번에 확인할 소규모 자료	긴 세션에서 중복·구식 내용 가능
경로/ID만 지시	경로·이슈 번호를 알려주고 *필요할 때 읽게 요청*한다.	긴 원문, 검색 가능 자료, 장기 프로젝트	실제 읽었는지 도구 기록·근거를 확인
자동 가져오기 (@...)	Claude Code `CLAUDE.md` 의 import는 참조 문서를 함께 컨텍스트에 적재한다.	매번 필요한 짧은 공통 지침	**자동 토큰 절약이 아니다.**[S1]
조건부 규칙	`.claude/rules/` 의 경로 조건에 맞을 때 해당 지침을 읽는다.	특정 파일 영역의 전문 규칙	규칙 매칭 및 로드 상태 점검 필요[S1]

경로를 '읽으라'고 적어도 실제로 읽었다는 보장은 없다. 확인이 중요하면 `열람 파일 → 적용한 조항 → 수행 결과` 를 보고하게 한다. 가능하면 도구 사용 기록이나 테스트로 교차 확인한다. 사소한 작업마다 장문의 사전 보고서를 요구하면 비용만 늘어난다.

실무용 '참조 카드'의 네 칸

[자료명] 파일 형식 정본
[위치] docs/spec/output-format.md
[언제] 파일 생성·검증·외부 제출 직전
[확인] 문서 버전·필수 필드·예시 출력의 불일치 여부

처음부터 복잡한 문서관리 시스템을 만들 필요가 없다. 이 카드 3장만으로 참조 능력을 연습할 수 있다.

작은 프로젝트와 성장한 프로젝트의 실전 파일 구조

시작 단계 — 목표 정본과 검증 예시만

my-project/
  README.md 무엇을 만들고 어떤 상태인가
  docs/requirements.md 사용자 목표와 합격 조건의 정본
  tests/ 실행 가능한 예시/테스트
  src/ AI가 적절히 구성하는 코드

장기 운영 단계 — 역할이 필요할 때만 분리

my-project/
  CLAUDE.md 상시 작업 규칙(짧게)
  README.md 목적·실행법·사용법
  docs/INDEX.md 지식·명세로 가는 얇은 안내판
  docs/requirements.md 목표·범위·인수 기준의 정본
  docs/architecture.md 중요한 설계 이유·경계
  docs/DECISIONS.md 결정의 배경·대안·날짜
  docs/STATUS.md 현재 상태·막힘·다음 조치
  docs/spec/ 세부 명세, 관련 작업만 열람
  src/ tests/ 구현 코드, 자동 검증
  evidence/ 개인·비밀정보 제외한 합격 증거
  .claude/rules/ 필요할 때만: 파일별 조건부 지침

작은 프로젝트에서 무리하게 모든 파일을 만들면 역효과다. `INDEX.md`에 본문을 중복 작성하지 않는다. 이슈 관리 도구를 쓴다면 `STATUS.md`는 필요 없다.

`CLAUDE.md`와 다른 파일의 역할

저장 장소	남겨야 할 정보	넣지 말아야 할 정보
`CLAUDE.md`	자주 다시 설명하는 프로젝트 규칙, 검증 명령, 출처·금지 경계	모든 진행 일지, 매번 바뀌는 작업 상태, 긴 API 문서
`docs/INDEX.md`	관련 원문의 **제목·경로·설명·찾는 조건**	원문의 전체 복제, 무조건 자동 import
`docs/requirements.md`	목표, 사용자, 기능, 범위, 인수 조건	일일 작업 진행상태
`docs/DECISIONS.md`	결론, 이유, 대안, 승인자, 적용일	아직 확인되지 않은 추정 사실을 확정처럼 기재
이슈 관리	할 일, 담당, 우선순위, 상태, 수용 기준, 증거 링크	길고 변하지 않는 기술 지식 원문
코드/테스트	프로그램의 실제 동작과 판정 방법	사람의 사업 의도를 자동으로 대체할 수 있다는 가정

사실 확인 · Claude Code의 `CLAUDE.md`

공식 문서는 `CLAUDE.md`를 200줄 미만으로 유지하라고 *권장*한다. `@경로` import는 구조화에 도움이 되지만 로드되는 정보량을 줄이지 않는다고도 적는다. 경로별 규칙과 하위 디렉터리의 지침은 상황에 따라 로드된다. `CLAUDE.md`는 행동을 유도하는 **문맥**이지 절대적인 접근 차단 기능이 아니다.[S1 S2]

짧고 검증 가능한 `CLAUDE.md` 예시

```
# 이 프로젝트의 작업 규칙

- 목표/인수 정본: docs/requirements.md
- 명세 색인: docs/INDEX.md. 작업에 관련된 원문만 실제로 읽는다.
- 작업 시작 시 변경 대상, 관련 요구사항, 검증 방법을 확인한다.
- 완료 전 프로젝트 테스트 명령을 실행하고 실패를 숨기지 않는다.
- 사용자 입력/데이터/권한에 영향을 주는 변경은 위험을 보고한다.
- 외부 배포·실데이터 삭제·결제·권한 확대는 승인 없이 실행하지 않는다.
- 진행상태는 이슈에, 중요한 결정의 이유는 docs/DECISIONS.md에 기록한다.
- 미검증 상태를 '완료'로 표시하지 않는다.
```

위의 승인 금지 문구는 **행동 지침**이다. 위험한 명령을 확실히 막으려면 Claude Code의 권한 설정·샌드박스·훅이나 외부 배포 승인 경계가 별도로 필요하다.[S2 S9]

'읽었다는 말'보다 나은 요청

이 작업의 정본은 docs/requirements.md이고 관련 세부 명세는
docs/INDEX.md에서 찾아라. 필요한 원문만 열어라.
수정 전, 실제로 참고한 문서 경로와 적용할 기준을 간단히 밝힌다.
관련 명세가 없거나 충돌하면 추측해 진행하지 말고 질문한다.
수정 후에는 해당 기준을 만족하는 테스트·출력 증거를 제시한다.

6장 대화는 작업 공간, 파일과 이슈는 공유 기억

긴 대화는 사람에게 흐름을 이어 준다. AI가 대화를 압축하거나 새 세션을 시작할 때 **원문·결정·테스트 결과가 자동으로 완벽히 보존되지는 않는다.** 대화와 프로젝트 상태를 분리한다.

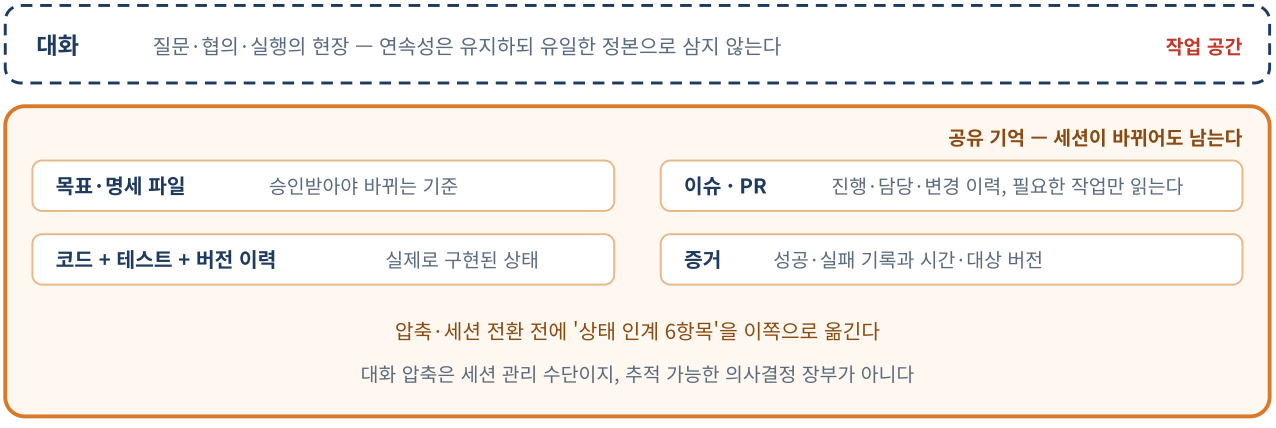


그림 5. 대화는 작업 공간, 파일·이슈·코드·증거는 공유 기억

프로젝트당 긴 대화 하나를 유지해도 된다. 프로젝트가 커져 여러 작업 세션을 병행하더라도 위의 외부 정본으로 지속성을 확보해야 한다.[S3]

압축·세션 전환 전 '상태 인계' 6항목

1. 현재 목표·범위·인수 기준의 정본 경로
2. 가장 최근 확인된 실행/테스트 상태와 버전
3. 진행 중인 이슈와 다음 우선순위
4. 아직 해소하지 못한 위험·가정·질문
5. 마지막으로 되돌릴 수 있는 안전한 버전 또는 복구 절차
6. 다음 세션에서 **먼저 읽을 최소한의 자료**

세션 정리 프롬프트

세션을 정리하자. 지금까지의 대화를 전체 요약하기보다
① 목표 정본 위치 ② 이번에 바뀐 확정 결정 ③ 통과한 테스트 증거
④ 실패/미완료 이슈 ⑤ 다음 행동 ⑥ 되돌릴 수 있는 버전을 정리해라.
영구 규칙이 아닌 일시 상태를 CLAUDE.md에 넣지 마라.
먼저 기존 파일·이슈와 대조해 충돌 여부를 보고하고 필요한 부분만 갱신해라.

제3부 · 실제로 작동하는 결과를 만드는 운영체계

7장 '만들었다'와 '합격했다'의 차이

소스 파일이 생성되었다는 말, 테스트가 통과했다는 말, 사용자가 제품을 인수했다는 말은 각각 다르다. **작업 완료 = 결과물 + 검증 증거 + 필요한 승인**이다. 하나라도 빠졌다면 '구현 완료 / 검증 대기'처럼 사실에 맞게 표시한다.



그림 6. 생성에서 인수까지 각 단계가 보장하는 것과 보장하지 않는 것

최초부터 자동화할 최소한의 검사 세트

1. 예시 입력을 넣으면 예상 형식의 파일이 생성되는가?
2. 필수 필드·개수·형식·값 범위가 맞는가?
3. 잘못된 입력을 넣으면 조용히 통과하지 않고 알아볼 수 있는 오류를 내는가?
4. 기능을 수정한 뒤 이전에 통과하던 테스트도 통과하는가?

5. 외부 시스템과 연동한다면 모의 환경과 실제 승인된 테스트 환경을 구분하는가?

Claude Code 공식 모범 사례도 **기계가 통과/실패를 판단할 수 있는 검사**가 있어야 에이전트가 스스로 수정하고 재검증할 수 있다고 강조한다.[S2]

검증 의뢰 프롬프트

```
이 기능을 완료했다고 선언하기 전에 다음 순서로 검증해라.
1. 현재 변경과 연결된 요구사항·합격 기준을 다시 읽는다.
2. 정상·경계·오류 입력을 포함한 테스트를 실행한다.
3. 기존 주요 기능에 대한 회귀 검증을 실행한다.
4. 실제 실행 화면/출력 파일/로그 중 가능한 증거를 제공한다.
5. 실행하지 못한 검증은 '미실행', 실패는 '실패'로 구분한다.
6. 남은 위험과 사람이 최종 인수할 사항을 적는다.
테스트를 통과시키기 위해 기준을 낮추거나 테스트를 삭제하지 마라.
```

8장 이슈는 작업 대장, 문서는 정본

장기 프로젝트에서 모든 `.md` 문서를 읽어야 하는 구조를 만들지 않는다. 일은 이슈에서 찾고, 자세한 내용은 문서를 정본으로 둔다. 이슈는 "업로드 결과가 형식 오류로 거부됨"으로 만들고, 형식 설명은 `docs/spec/output-format.md`에 둔다. PR(변경 제안)은 어떤 변경이 코드에 실제로 반영됐는지 추적한다.

권장 탐색 순서는 ① 프로젝트 목표와 현재 상태의 짧은 안내판 확인 → ② 관련된 이슈 제목·상태 검색 → ③ 해당 이슈의 완료 조건·담당·선행관계 읽기 → ④ 이슈에 연결된 필요한 원문만 열람 → ⑤ 구현·검증 후 증거 링크와 상태 갱신이다.

최소 이슈 양식

```
## 목적
왜 이 작업이 필요한가?

## 기대 결과물
무엇이 실제로 생성/변경되는가?

## 범위와 제외
바꿀 것 / 바꾸지 않을 것

## 선행 작업·참조 정본
이슈 번호와 원문 경로

## 완료 기준
- [ ] 정상 사례 통과
- [ ] 오류/경계 사례 통과
- [ ] 회귀 검사 통과
- [ ] 결과물과 증거 링크 첨부

## 담당 및 위험
주담당 / 영향 범위 / 필요한 승인
```

이슈 시스템이 없다면 간단한 작업 목록 파일로 시작해도 된다. 도구를 늘리기 전에 조회·검색·완료 확인이 실제로 편해지는지 먼저 판단한다.

9장 자율성과 안전을 동시에 높이는 최소 통제

AI의 모든 선택을 승인받게 하지 않는다. 위험을 기준으로 **세 수준의 경계**를 두고, 사람은 **다섯 개의 레버**(목표·범위·권한·검증·중단)만 잡는다.

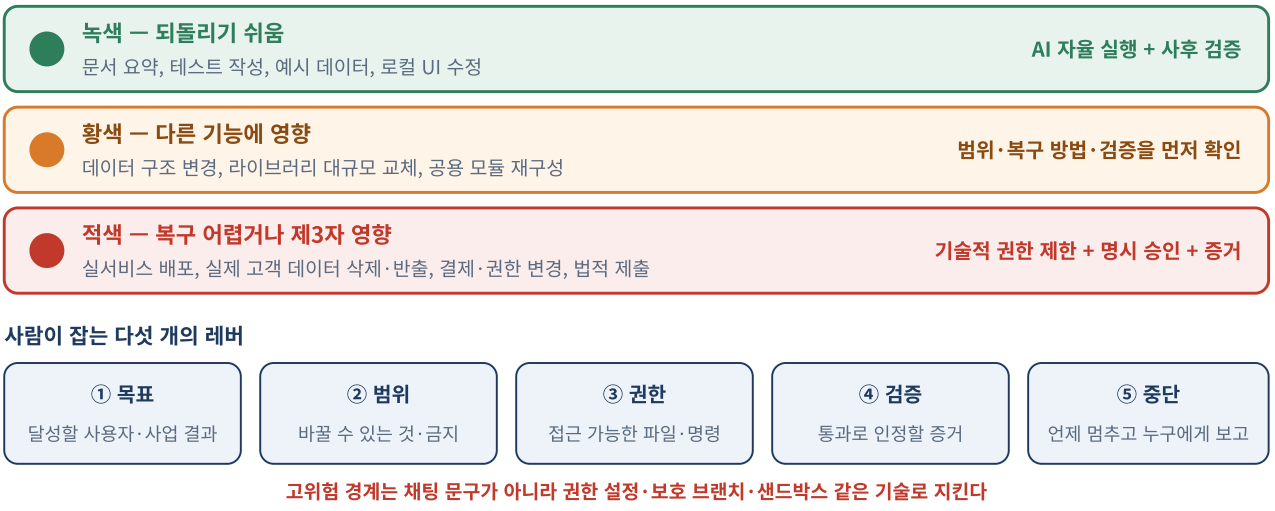


그림 7. 위험 기준 세 수준의 경계와 사람이 잡는 다섯 레버

이 색 분류는 **운영 판단 모델**이지 특정 도구에 구현된 보안 기능이 아니다. 실제 통제는 Git 보호 규칙, 접근 권한, 승인 절차, 샌드박스, 권한 설정, 자동 검사로 한다. 각 레버를 수십 개의 지시로 세분화할 필요는 없다. **고위험 경계는 채팅 문구만으로 보호하지 않는다.**[S2 S6 S7 S9 S10]

무인 실행을 위한 최소 네 조건



그림 8. 무인 실행의 네 조건

사실 확인 · Claude Code의 '사람 없이 실행하기'

Claude Code 공식 모범 사례는 무인 실행 전에 세 가지를 확인하라고 한다. 완료가 무엇인지 아는가, 스스로 검사할 수 있는가, 건드려야 할 것만 건드리는가. 도구로는 완료 조건을 매 턴 확인하는 `/goal`, 작업을 끝내려 할 때 검사를 강제하는 `Stop` 훅, 턴·비용 상한, 마지막 사람 승인을 든다. 권한의 `deny` 규칙은 어떤 모드에서도 유지된다. 반드시 일어나야 할 행동은 `CLAUDE.md` 가 아니라 훅이나 관리 설정에 둔다. 이 책의 네 조건은 그 권고를 비개발자의 언어로 옮긴 것이다.[S1 S2 S9]

안전한 중단 조건의 설계 예시

다음 값은 프로젝트별로 조정하는 초기값이다.

- 동일 유형의 오류가 **3회 연속** 재발하면 원인 분석 후 중단
- 당일 허용 비용·실행 시간 중 하나가 한도에 도달하면 중단
- 통과 테스트가 예상치 않게 감소하면 변경 중단
- 외부 서비스에서 인증 실패·권한 확대·과금 요청이 발생하면 중단
- 합격 기준을 충족하지 못했는데 '성공'이라고 표시하면 인수 중단
- 외부 배포·실데이터 삭제가 필요한 경우 승인 없이는 중단

'중단하라'는 프롬프트만으로 자동 중단이 보장되지는 않는다. 실행 도구의 실제 제한 설정과 감독 장치를 확인한다.

10장 작은 수정을 안전하게 잇는 복구 가능한 변경

Git 명령어를 먼저 암기할 필요는 없다. 다음 개념을 익힌다.

개념	쉬운 해석	반드시 확인할 질문
버전 기록(커밋)	잘 작동하던 시점의 스냅샷	마지막 정상 버전은 어디인가?
작업 분기(브랜치)	원본을 망가뜨리지 않는 시험장	실서비스 작업과 섞이지 않았는가?
변경 제안(PR)	이 변경을 정식 반영해도 되는지 심사	무엇이 바뀌고 무엇을 시험했는가?
병합	검토된 변경을 본선에 반영	자동 검사·리뷰·승인 조건이 충족됐는가?
되돌리기	안전한 지점으로 복귀	데이터 변경까지 되돌릴 수 있는가?

코드 변경을 되돌리는 것과 이미 삭제된 실데이터·외부 서비스에 보낸 요청을 되돌리는 것은 **다르다**. 코드 버전만으로 모든 복구가 되지는 않는다.[S10]

11장 도구는 수준을 결정하지 않는다

사용 환경	강점	권장 시작점	꼭 알아둘 제한
일반 채팅형 AI	요구사항 토론·분해·검토에 쉬움	L0~L1	실제 파일 실행/수정 권한은 환경별로 다름
웹 기반 AI 빌더	빠른 화면·프로토타입	L0~L2	코드·데이터·배포·복구의 소유권 확인
Claude Code	프로젝트 파일 읽기·편집·실행·테스트	L1~L4	권한 설정·프로젝트 루트·정본 관리 필요
Git/이슈 도구	작업·변경·결정 추적	L2~L5	관리 도구만 늘어도 제품이 완성되지는 않음
OMC	다중 에이전트 실행·반복·역할 분화	**L3~L5에서 시험 도입**	플러그인 버전·설정·추가 비용·권한 위험 점검

OMC(oh-my-claudecode)는 Claude Code 위에서 에이전트와 작업 흐름을 조율하는 오픈소스 프로젝트다. **기존의 목표·작업·검증 체계를 자동화하는 상위 실행 계층**이며, 새로운 개발 원리를 대체하지는 않는다. 아래 명령 이름은 2026년 11월 9일 기준 공식 저장소 문서와 대조했다. 이후 바뀔 수 있다.[S4 S5]

12장 OMC를 위한 단계별 학습 경로

0단계 · OMC 없이 기준선 확보

동일한 작은 작업을 기본 Claude Code로 수행한다. 무엇을 완성했는지, 실제로 통과한 검사 수와 소요 시간·비용은 얼마인지 기록한다. 기준선이 없으면 OMC가 도움이 됐는지 판단할 수 없다.

1단계 · 문제를 분명하게 만들기

요구사항이 흐릿하다면 `/deep-interview`처럼 질문으로 요구사항을 구체화하는 기능을 사용한다. 인터뷰 결과도 사용자 의도·제약·인수 기준의 정본으로 확인해야 한다.

```
/deep-interview "업무용 데이터 변환기를 만들고 싶다.
현재 내가 모르는 전제와 합격 기준을 질문으로 찾아줘."
```

2단계 · 계획하고 작은 작업을 실행

계획 기능(`/ralplan`, 반복 합의형 계획)으로 작업 분해안을 만들고, **한 작업에 한 담당·한 검증**을 붙인다. 사용 중인 설치 버전의 도움말을 먼저 확인한다.

3단계 · 자율 실행을 제한된 범위에서 시험

`/autopilot`은 아이디어에서 구현까지 단일 주도 에이전트가 자동으로 진행하는 계열이다. **정본·검증 기준·비용/시간 한도·중단 경계**가 준비됐을 때 쓴다. 제품 전체를 처음부터 무제한 맡기지 않는다.

```
/autopilot "샘플 데이터 3개를 받아 지정된 형식의 파일을 생성하는
로컬 프로토타입을 만들어라. 실제 외부 서비스 호출은 금지한다.
명세는 docs/requirements.md를 따르고 자동 검사 통과 증거를 남겨라."
```

4단계 · 반복 검증과 병렬 실행을 분리

기능 계열	주된 목적	언제 유익한가	사용하지 말아야 할 때
`/deep-interview`	모호한 요구 구체화	기획 불확실성이 큰 시작점	이미 검증된 명세가 충분한 경우
`/ralplan`	작업의 순서·경계 검토	의존성·위험이 복잡한 변경	사소한 문구 수정
`/execute`	합의된 작업 구현·검증	작은 범위의 구현	목표·조건이 모호한 경우
`/autopilot`	넓은 범위 자율 실행	조건과 종료 판정이 갖춰진 프로토타입	고위험 외부 행동이 열려 있는 경우
`/ralph`	검증될 때까지 지속 반복	명확한 단일 완료 조건·안전한 한도	완료 조건이 주관적이거나 무제한 반복 위험
`/team 3:executor`	독립 작업의 병렬 분담	셋 이상으로 분리되는 독립 작업과 소유자	같은 파일/데이터를 동시에 수정하거나 조정비용이 큰 경우
`/goal` (Claude Code 기본 기능)	측정 가능한 완료 조건 하나를 여러 턴에 걸쳐 추적	OMC 루프를 쓰지 않는 단순 작업	Ralph·Team·Autopilot이 이미 루프를 이끄는 경우

팀 모드는 에이전트 숫자가 많다는 이유만으로 빠르거나 싸지 않다. 작업 간 독립성, 합치기 비용, 중복 조사, 상충 수정, 인수 증거를 함께 측정해야 한다. **지속 실행을 지휘하는 기능은 세션당 하나만** 켠다. OMC의 모드 선택 가이드는 Autopilot·Ralph·Team과 Claude Code의 `/goal` 중 하나를 주 제어 주체로 정하고 경쟁하는 지속 루프를 동시에 돌리지 말라고 안내한다. 중복 제어는 종료 조건과 책임을 흐린다.[S5]

설치와 보안 점검

- 먼저 **공식 저장소**(github.com/Yeachan-Heo/oh-my-claudecode, MIT 라이선스)와 변경 이력을 확인한다.
- 설치는 Claude Code 플러그인 마켓플레이스가 기본이다. 대안으로 npm CLI(패키지 `oh-my-claude-sisyphus`, 명령 `omc`)가 있다. **하나만** 선택한다. 두 방식을 함께 쓸 때는 스킬·에이전트가 중복 설치되지 않는지 점검한다.
- 타사 플러그인이 설치하는 명령·훅·권한 규칙·외부 도구 연결의 범위를 확인한다. OMC의 `omc lookout scan` 같은 사전 위험 점검 기능이 있다면 실행 전에 쓴다.
- 실고객 데이터나 운영 시스템에 접근하지 않는 별도 실험 저장소에서 시작한다.

- 기본 Claude Code의 동일 작업과 정확성·시간·토큰/비용·복구 난이도를 비교한다.
- 대화 안의 `/...` 스킬과 터미널의 `omc ...` 명령을 구분한다. 명령·버전 정보는 최신 문서로 재검증한다.[S4]

OMC 도입 여부를 평가하는 작은 실험

동일한 평가 기준으로 3회씩 시행한다.

평가	기본 Claude Code	Claude Code + OMC
요구사항 충족 수 / 전체 수	기록	기록
자동 테스트 통과 수 / 전체 수	기록	기록
외부 인수 합격 여부	기록	기록
재작업·사람 개입 횟수	기록	기록
총 수행 시간	기록	기록
모델·도구 사용비용	기록	기록
실수·보안 문제·복구 비용	기록	기록

OMC를 써서 로그나 PR이 늘어난 것은 **성공이 아니다**. *동일한 품질과 위험 수준에서 순시간·순비용을 줄이거나, 같은 자원으로 더 많은 합격 결과를 낸 것*을 확인해야 성공이다.

13장 비개발자가 반드시 알아야 할 권한과 보안

보안은 'AI에게 조심하라고 말하기'가 아니라 *못 해야 하는 행동을 실제로 못 하게 만드는 장치*다.

위험	예	대응 원칙
비밀키 노출	`.env`, 접근 토큰, 서명키가 모델 입력·로그에 포함	비밀 저장소 분리, 접근 차단, 유출 시 즉시 회전
외부 문서의 악성 지시	웹페이지·이슈 내용에 "명령 실행" 삽입	원문을 **자료**로만 취급, 도구 실행 명령으로 승격 금지
권한 과다	읽기 전용 조사 에이전트가 배포·삭제 가능	최소 권한, 검토 역할은 읽기 전용
의존성·플러그인 위험	패키지 설치 스크립트·훅이 명령 실행	출처·변경 파일·설치 과정 검토
검증 조작	실패하는 테스트를 삭제하거나 기준 낮춤	테스트 변경 자체를 심사, 외부 독립 검증 유지
범위 이탈	요청하지 않은 파일이나 CI 설정 변경	변경 목록과 승인 경계 비교
공유 환경 충돌	다중 에이전트가 같은 파일 수정	소유권/브랜치 분리, 순차 병합·재검증

OWASP는 AI 보조 코딩에서 비밀정보 유출, 프롬프트 인젝션, 공급망, 규칙 파일 변조, 테스트 조작, 에이전트 간 악성 지시 전파를 주요 위험으로 다룬다. `.gitignore`에 넣었다고 AI가 파일을 읽지 못하는 것은 아니다.[S6]

비개발자 최소 요구사항

1. 실데이터 대신 가짜 샘플 데이터로 첫 구현을 한다.
2. 비밀키를 채팅·공유 문서·소스 코드에 붙여넣지 않는다.
3. 배포·삭제·과금·권한 확대는 승인 경계를 둔다.
4. 코드 수정자와 검증자의 관점 또는 권한을 분리한다.
5. 외부 사용자가 접근하는 서비스, 결제/개인정보 처리는 전문 개발·보안 검토를 필수 단계로 잡는다.

`CLAUDE.md`에 '하지 말라'고 쓴 것과 실제 `permissions.deny`, 샌드박스, 보호 브랜치 같은 강제 제한은 구별해야 한다. 설정 예시는 도구 버전에 따라 달라진다.[S2 S6 S7 S9]

제5부 · 실행 가능한 교육 과정

14장 12주 성장 실습 로드맵

기간은 강제가 아니라 학습 속도를 맞추는 기준이다. 각 단계의 **통과 증거**를 얻지 못하면 다음 도구를 설치하기보다 같은 역량을 다른 작은 과제로 반복한다.



단계별 코칭 질문

L0→L1 "원하는 결과를 예시와 실패 사례까지 보여줄 수 있는가?" · L1→L2 "내가 자리를 비워도 AI가 정확한 정본을 찾을 수 있는가?"
L2→L3 "AI가 성공했다고 할 때 거짓 완료를 발견할 방법이 있는가?" · L3→L4 "한 주 뒤 다른 사람이 작업의 상태·담당·증거를 이해하는가?"
L4→L5 "사람 또는 에이전트 하나를 더 늘렸을 때 실제 순효익이 증가했는가?"

그림 9. 12주 로드맵의 주차별 과제와 단계 사이의 코칭 질문

주차	훈련 과제	반드시 남길 결과물	통과 조건
1주	작은 계산기·파일 변환기 목표 정의	1쪽 목표/수용 기준	다른 사람이 성공 여부를 동일하게 판정
2주	AI가 분해한 3~7작업 비판	분해표·누락/중복 수정	모든 작업에 결과·검증이 존재
3주	정상·오류 입력 실험	3개 이상 입력/출력·실패 예시	실패도 예측 가능한 메시지로 처리
4주	경로 기반 자료 참조	요구사항 정본·색인 카드	새 세션에서 같은 기준을 재현

주차	훈련 과제	반드시 남길 결과물	통과 조건
5주	Claude Code 프로젝트 작업	짧은 `CLAUDE.md`	지침과 실제 도구 제한의 차이를 설명
6주	자동 테스트·회귀 확인	테스트 실행 기록	수정 후 이전 기능도 통과
7주	Git/이슈·PR	이슈 3건, 변경 1건	변경과 합격 증거를 역추적
8주	의도적인 오류·복구	실패 이슈·수정·복구 기록	마지막 정상 상태 복원 가능
9주	제한된 자율 작업	실행 한도·중단 조건·로그	무인 1개 작업의 증거 기반 종료
10주	OMC 요구 인터뷰·작은 실행	기본 Claude Code 대비표	추가 사용의 이득/손실 설명
11주	두 독립 작업만 병렬화	소유권 표·병합/검증 기록	충돌 없이 통합 테스트 통과
12주	실제 업무형 인수 시험	인수 체크리스트·실행 결과	사용자가 정한 최종 조건 충족

15장 세 가지 학습 시나리오

A. 오늘만 쓰는 엑셀/CSV 변환기

입력 예시 2개와 기대 출력 예시 2개를 첨부한다 → 형식 오류 시 처리 방법을 정한다 → AI에게 구현·실행·출력 파일 비교를 맡긴다 → 테스트 결과와 실제 변환 파일을 열어 확인한다. **권장 구조**: 긴 목차는 필요 없다. 입력/출력 예시와 검증이 핵심이다.

B. 몇 달간 발전시킬 사내 대시보드

사용자별 화면·권한·핵심 지표·변경 빈도를 구분한다 → 요구사항 정본·얇은 색인·작업 이슈를 마련한다 → 기능 단위로 변경하고 자동 테스트·수동 인수를 섞는다 → OMC 팀 도입은 독립된 기능과 검증 체계가 있을 때만 시험한다. **권장 구조**: `CLAUDE.md`는 짧게 쓰고, 상세 데이터 정의는 별도 문서에, 진행은 이슈에 둔다.

C. 외부 사이트에서 합격받아야 하는 파일 생성 시스템

공식 형식 명세·제약·예시를 우선 확보한다 → 생성 파일을 **로컬 검증기**로 검사한다 → 외부 업로드에 앞서 중복 제출·요금·개인정보 위험을 확인한다 → 승인된 테스트 환경에서 업로드 후 실제 접수·반려·합격 응답을 저장한다 → 합격이 나지 않으면 '완료'가 아니라 '외부 검증 대기/실패'로 표시한다. **권장 구조**: 최종 완료 기준은 **제3자 시스템의 합격 증거**다.

16장 문제를 발견했을 때의 증상·원인 가설·즉시 조치

관찰된 증상	가장 먼저 의심할 원인	추천 조치
전날 결정한 기준이 달라졌다	압축·대화 의존·상충 정본	목표 정본과 결정 이력을 찾아 비교
관련 자료를 읽지 않은 듯하다	경로 오류·검색 실패·읽기 누락	실제 읽은 파일과 근거 조항 확인
`CLAUDE.md`가 너무 길어졌다	일지·원문·규칙 혼재	영구 규칙만 남기고 이슈·문서로 이동

관찰된 증상	가장 먼저 의심할 원인	추천 조치
수정하면 다른 기능이 망가진다	영향 범위 미파악·회귀 테스트 부재	변경 경계 재확인, 이전 테스트 추가
PR이 많은데 제품이 없다	활동량을 완료로 착각	사용자 시나리오의 실제 실행·인수 요구
AI가 같은 오류를 계속 고친다	완료 조건 불명확·진단 없는 반복	반복 중단, 실패 재현부터 다시 시작
팀 에이전트가 서로 파일을 덮어쓴다	작업 소유권·브랜치 경계 불명확	병렬 정지, 독립 작업 재분할
토큰/비용이 늘지만 성과가 없다	불필요한 읽기·중복 에이전트·재작업	단일 기준선과 비교, 정보·역할 축소
외부 배포 후 문제가 생겼다	승인/테스트/복구 장치 미흡	영향 확산 중단, 배포·데이터 복구 절차 발동
AI가 근거 없이 "100% 완료"라고 한다	검사 불가능한 목표·자기 보고 의존	재현 가능한 테스트·제3자 결과 요구

제6부 · 즉시 복사해서 쓰는 서식

17장 템플릿 아홉 가지

AI에게 주는 순서

- ① **이 PDF 파일을 AI 대화창에 드래그 앤 드롭해 첨부한다.** 템플릿 안의 용어(정본, 합격 기준, 경계, 증거)를 AI가 같은 뜻으로 읽게 하기 위해서다.
- ② **필요한 템플릿을 복사해 빈칸을 채우고 보낸다.** 모든 작업에 모든 템플릿을 쓰지 않는다. 권한·구조·목표·비용에 큰 영향이 있는 일에 집중한다.
- ③ **같은 내용의 AI용 MD 파일을 받았다면**, 그 파일을 붙여 넣거나 첨부하고 엔터만 눌러도 AI가 단계 진단부터 템플릿 작성까지 안내한다.

템플릿 A · 프로젝트 발주서 (1페이지)

```
# 프로젝트 발주서

## 목적과 사용자
- 문제:
- 사용자:
- 이 프로젝트가 해결하는 일:

## 최종 결과물
- 결과물 형식(앱, 파일, API, 서비스 등):
- 입력 예시:
- 출력 예시:
- 최종 외부 인수 또는 합격 판정:

## 필수 기능과 제외 범위
- 반드시 동작할 것:
- 이번에는 만들지 않을 것:

## 완료 기준
- 정상 사례:
- 오류·경계 사례:
- 성능/형식/접근성 등 필수 조건:
- 증거(테스트, 파일, 화면, 합격 ID):

## 제약과 통제
- 예산/시간 한도:
- 보호할 데이터·권한:
- AI 자율 판단 허용 범위:
- 반드시 내 승인이 필요한 일:
- 자동 중단 조건:
```

템플릿 B · AI의 분해안을 검토하는 표

ID	작업 결과물	선행조건	합격 기준	위험	담당	상태
T1						

누락된 사용자 흐름:
중복되는 작업:
검증할 수 없는 작업:
내가 결정해야 하는 쟁점:
AI에게 맡길 기술적 구조화:

템플릿 C · 자료 참조 색인

지식 색인

자료	실제 경로/이슈	언제 읽나	상태/최신일
목표 및 합격 기준	docs/requirements.md	모든 작업 시작 시	승인/날짜
출력 형식	docs/spec/output-format.md	출력 변경·제출 전	승인/날짜
결정 이유	docs/DECISIONS.md	기존 설계를 바꾸기 전	날짜

규칙: 필요 문서만 읽고, 존재하지 않으면 추측하지 않는다.

템플릿 D · 변경 전/후 보고

[변경 전]
- 관련 이슈와 정본:
- 현재 동작:
- 바꾸려는 범위/영향받는 기능:
- 실패 위험과 되돌릴 방법:
- 실행할 검증:

[변경 후]
- 실제 바뀐 파일/기능:
- 실행한 검사와 통과/실패/미실행:
- 증거 링크 또는 출력:
- 남은 위험:
- 다음 단계/필요한 사람 승인:

템플릿 E · 의사결정 요약 (중요한 결정에만)

결정할 사항:
현재 근거와 자료 위치:
가능한 대안(2~3개):
선택안과 이유:
불확실성 또는 확인되지 않은 가정:
위험 및 되돌리기:
누가 승인해야 하는가:
다음 행동:

템플릿 F · 회귀·인수 체크리스트

```
## 검사 대상
- 대상 기능/버전:
- 기준 문서:

## 결과
- [ ] 정상 입력 A 통과 — 증거:
- [ ] 정상 입력 B 통과 — 증거:
- [ ] 오류 입력 C 기대대로 거부 — 증거:
- [ ] 기존 핵심 기능 회귀 검사 통과 — 증거:
- [ ] 실제 사용 시나리오 통과 — 증거:
- [ ] 외부 사이트 판정 통과(해당 시) — 증거:
- [ ] 개인정보/권한/비밀정보 점검 — 증거:
- [ ] 미검증 항목과 제한 명시 — 근거:

판정: [인수 / 조건부 인수 / 반려 / 검증 대기]
판정자 및 날짜:
```

템플릿 G · 무인 실행 요청서

```
이번 실행의 목표는 [작고 재현 가능한 결과]다.
정본: [경로/버전]
허용 작업: [로컬/테스트 환경의 범위]
금지 작업: [실서비스 배포, 데이터 삭제, 과금, 권한 확대 등]
합격 판정: [검사 명령/예시 입력과 출력/외부 응답]
기록 위치: [이슈, 로그, 결과 파일]
중단 조건: [연속 실패 수, 예산, 시간, 이상행동]
복구 기준점: [안전한 버전과 절차]

위 제한 중 기술적으로 집행할 수 없는 항목이 있으면 실행 전에 구분하여 보고해라.
검증되지 않은 상태를 완료로 선언하지 말고, 종료 시 성공/실패/미실행을
증거와 함께 분리해라.
```

템플릿 H · OMC 실험 기록

실험 목표:
완료 기준:
동일한 비교 과제:
기본 Claude Code 결과: 시간 / 비용 / 검증 통과 / 개입 / 사고
OMC 결과: 시간 / 비용 / 검증 통과 / 개입 / 사고
OMC 기능·설정·버전:
다중 에이전트 조정·병합에 추가된 비용:
순효익 판단:
다음 시험에서 유지/변경할 조건:

템플릿 I · 1주 회고

이번 주에 실제로 합격한 결과물:
검증 없이 넘어간 일:
가장 비싼 재작업과 원인:
AI에게 더 넓게 위임해도 될 일:
사람 또는 기술적 장치의 통제가 필요한 일:
이번 주 새로 익힌 구조화/참조 습관 하나:
다음 주 개선할 습관 딱 하나:

제7부 · 성과 측정과 향후 운영

18장 토큰보다 중요한 지표

토큰 사용량과 토큰당 ROI(투자수익률)는 운영 효율을 파악하는 **보조 지표**로 유용하다. 쉬운 과제를 많이 수행하거나 비싼 모델을 많이 사용하는 것이 곧 가치 창출은 아니다.

지표	계산·관찰 방법	주의점
검증 완료율	증거가 있는 합격 작업 / 전체 완료 주장 작업	테스트를 임의로 축소하지 않았는지 검토
첫 통과율	최초 제출에서 통과 / 전체 제출	어려운 일을 피하면 높아질 수 있음
재작업 비용	반복 수정에 소요된 사람 시간 + AI 비용	숨은 QA·조정 비용 포함
리드타임	요구 승인부터 인수까지 걸린 시간	대기·외부 승인 시간 구분
안전한 자율 실행 비율	기준·로그·복구를 갖춘 자율 작업 / 대상 작업	자율성 확대 자체가 목적이 아님
토큰/비용당 합격 결과	검증된 성과량 / AI 비용	기능 난이도·품질·위험 차이 보정 필요
사업 순가치	추가 매출 또는 절감된 총비용 − AI·인건비·운영·사고 비용	인과관계와 비교 기준선 확보

측정은 30/60/90일마다 한다. 동일한 유형의 작업을 `수작업 / 단일 AI / 사람이 검토하는 AI / 여러 AI 에이전트`로 비교하고, 작업 난이도와 사람의 개입 시간을 기록한다. 경쟁적으로 토큰을 많이 쓰도록 보상하지 않는다. 어떤 환경에서는 AI로 시간이 단축되고, 다른 환경에서는 검증·재작업 때문에 오히려 느려질 수 있다.

사실 확인 · AI가 늘 빠르게 만드는 것은 아니다

METR이 2025년 7월에 발표한 무작위 대조 실험(숙련 오픈소스 개발자 16명, 실제 이슈 246건)에서는 AI 도구를 쓴 쪽이 오히려 **19% 느려졌다**. 참가자들은 실험 전에 24% 빨라질 것으로 예상했고, 끝난 뒤에도 20% 빨라졌다고 느꼈다. 연구진은 표본이 작고 일반화에 한계가 있다고 밝혔다. 2026년 2월 후속 글에서는 두 번째 실험의 자료가 신뢰하기 어려워 실험 설계를 바꾸겠다고 알렸다. 새 수치를 낸 것은 아니다. 이 결과가 모든 환경에 그대로 적용된다는 뜻은 아니다. **자기 환경의 기준선을 직접 재야 한다**는 뜻이다.[S8]

19장 초보자에게 남기고 싶은 최종 기준

작은 일에도 습관을, 큰 일에는 시스템을

1. **처음부터 최고 수준의 기준을 배운다.** 목적·구조화·참조·검증·책임의 원칙은 초급에서도 똑같다.
2. **처음부터 최고 수준의 복잡성을 도입하지 않는다.** 조건부 규칙·다중 에이전트·자동 훅은 필요와 효과가 확인될 때 추가한다.
3. **AI가 분해하도록 하되, 사람이 분해를 검토하는 능력을 기른다.** 이것이 비개발자의 장기 경쟁력이다.
4. **AI의 자율성은 지시문 길이가 아니라, 정본·검증·권한·복구가 결정한다.**
5. **'많이 만들었다'보다 '검증되어 인수되었다'를 성공으로 삼는다.**
6. **에이전트 숫자보다 인간과 AI가 함께 만든 순증분 가치를 평가한다.**

> 한 문장 운영 철학 — AI에게 구현의 자유를 주되, 목표는 정본으로 남기고, 작업은 구조화하며, 결과는 증거로 검증하고, 위험은 기술적으로 제한한다.

부록 A 근거 자료(2026년 11월 9일 원문 확인)

인용은 원칙·기능·보안 조언을 뒷받침하는 **공식 제공자 문서·프로젝트 원본·1차 연구**를 우선했다. 본문의 S# 표시가 아래 번호다. 제품 기능은 빠르게 바뀌므로 실제 명령어 사용 직전에 최신 문서를 다시 확인한다.

- **[S1]** Anthropic, *How Claude remembers your project* (Claude Code 문서) · https://code.claude.com/docs/en/memory · `CLAUDE.md` 의 목적, 200줄 미만 권고, `@경로` import의 동작, 경로별 규칙(`.claude/rules/`), 지침과 강제 통제의 차이(반드시 일어나야 할 행동은 훅·관리 설정으로).
- **[S2]** Anthropic, *Best practices for Claude Code* · https://code.claude.com/docs/en/best-practices · 검증 수단 제공, 증거 요구, 무인 실행의 종료 조건·훅·검증 하위 에이전트, 비용·문맥 관리.
- **[S3]** Anthropic, *Troubleshooting* (Claude Code 문서) · https://code.claude.com/docs/en/troubleshooting · 세션·컨텍스트·플러그인 문제의 운영 조언.
- **[S4]** oh-my-claudecode 공식 저장소 · https://github.com/Yeachan-Heo/oh-my-claudecode · MIT 라이선스. 플러그인 마켓플레이스 설치와 npm CLI(`omc`), `/deep-interview` · `/ralplan` · `/execute` · `/autopilot` · `/ralph` · `/team` 스킬, `omc lookout scan` .
- **[S5]** oh-my-claudecode, *Mode Selection Guide* · https://github.com/Yeachan-Heo/oh-my-claudecode/blob/main/docs/shared/mode-selection-guide.md · Autopilot·Ralph·Team의 선택 기준, Claude Code `/goal` 과의 관계, 지속 루프 중복 운용 금지.
- **[S6]** OWASP, *Secure Coding with AI Cheat Sheet* · https://cheatsheetseries.owasp.org/cheatsheets/Secure_Coding_with_AI_Cheat_Sheet.html · 비밀정보·외부 입력·규칙 파일·테스트 조작·다중 에이전트 보안 위험.
- **[S7]** NIST, *Secure Software Development Framework (SSDF) SP 800-218 v1.1* (2022-02-03) · https://csrc.nist.gov/pubs/sp/800/218/final · 보안 개발·검증·위험 관리의 실천 틀. 법적 적합성 인증은 아니다. 생성형 AI용 보완판 *SP 800-218A* (2024-07-26) · https://csrc.nist.gov/pubs/sp/800/218/a/final
- **[S8]** METR, *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity* (2025-07-10) · https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/ · 후속 *We are Changing our Developer Productivity Experiment Design* (2026-02-24) · https://metr.org/blog/2026-02-24-uplift-update/
- **[S9]** Anthropic, *Configure permissions* (Claude Code 문서) · https://code.claude.com/docs/en/permissions · 허용·확인·거부 규칙, 권한 모드, 샌드박스.
- **[S10]** GitHub Docs, *About protected branches* · https://docs.github.com/en/repositories/configuring-branches-and-merges-in-your-repository/managing-protected-branches/about-protected-branches · 병합 전 검사·승인·보호 규칙의 일반 원리. Gitea 등 다른 도구의 설정과는 구분한다.

근거 수준과 한계

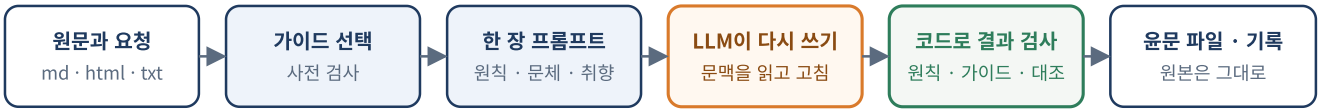
- **공식 문서로 확인한 것**은 `CLAUDE.md` 의 역할과 import 방식, Claude Code 검증·권한 지침, OMC 공개 기능 계열과 설치 경로, OWASP 보안 위험, METR 수치다.
- **이 책의 교육적 설계**는 L0~L5 성장 수준, 5분 루틴, 12주 로드맵, 작업 분해 검토 문항, 위험 색 분류, 이슈·문서 폴더 예시다. 검증된 국제 표준이나 제품 기본 설정이 아니라 실무 적용을 위한 제안이다.
- **의도적으로 제외한 것**은 출처와 시점이 불명확한 성공담, 확증하기 어려운 매출 수치, "N명 에이전트는 X배 빠르다" 같은 일반화다.
- **실측하지 않은 것**은 독자의 실제 Claude Code·OMC 설치 환경, 저장소, 외부 합격 시스템이다. 해당 환경의 설정·비용·보안 적합성은 이 문서만으로 보증할 수 없다.

문서 버전 2026-11-10 윤문판 · 본문 문장을 업무 문체 기준으로 다듬음, 부록 B(윤문 도구 소개) 추가. 그림·표·템플릿·출처·사실 관계는 2026-11-09판과 같다. 2026-11-09판의 변경: 출처 10건 원문 재확인(`/ralplan` , `/goal` 소속 정정), 「흔한 오해와 바로잡기」 전환, 반복 경고 통합, 도식 9장·템플릿 순서 추가, AI 붙여넣기용 MD 동봉.

부록 B · 이 책에 쓴 윤문 도구

이 책의 문장은 slop-rewrite가 다듬었습니다

slop-rewrite는 AI가 쓴 한국어 초안에서 AI 티를 덜어 내고, 장르와 문체, 쓰는 사람의 취향에 맞게 다시 쓰는 오픈소스 스킬입니다. Claude Code와 Codex에서 **「윤문해줘」「퇴고해줘」「AI 티 없애줘」**라고 말하면 움직입니다. 코드가 고칠 자리를 짚고, LLM이 문맥을 읽어 문장을 다시 쓰고, 코드가 결과를 다시 검사합니다. 사람이 읽기에 자연스럽고 핵심이 곧장 전달되는 한국어가 첫째 기준입니다.



검사에 걸리면 담당이 한 번 고치고 다시 검사한다 · 통과 못 한 자리는 작업 기록에 남는다

그림 10. slop-rewrite가 한 편을 고치는 순서

세 레이어로 맞춘다

- **원칙**: 누구에게나 AI 티인 것 67규칙. 줄표 남용, 이항 대구 연쇄, 3항 강박, 요약 마무리 같은 것
- **스타일가이드**: 장피엠·업무·토스·개조식 가운데 고르고, 내 글로 내 가이드를 만들어 등록
- **취향**: 결과물에 단 「내가 쓴다면 이렇게」 코멘트가 다음 윤문의 규칙이 된다

고치면서 지키는 것

- 원문에 없는 숫자·이름·인용·출처·주장을 보태지 않는다
- 남긴 주장의 부정·조건·시제·확정 수준·단위를 바꾸지 않는다
- 표·코드·인용문·템플릿의 고정 구역은 그대로 둔다
- 원본 파일은 손대지 않고 결과 파일과 작업 기록을 따로 만든다

이 책에 적용한 결과

- 가이드 「업무」로 본문 77곳을 다듬음: 가이드 18 · 원칙 47 · 사람 판단 12
- 그림 9장 · 표 15개 · 템플릿 21개 · 출처 10건은 한 글자도 바뀌지 않음
- 검수 통과: 원칙 S1 0 · S2 0, 원문 대조 S1 0
- 고친 것 · 뺀 것 · 확신 낮은 곳이 작업 기록에 남아 발행자가 하나씩 확인

시작하기

```
git clone https://github.com/byungjunjang/slop-rewrite.git
```

1. **바로 써 보기**: Claude Code 또는 Codex에서 이 폴더를 열고 파일을 지정해 「이 글 윤문해줘」라고 한다. 원본 옆에 `원본명.taste.md` `원본명.taste-notes.md` 가 생긴다.
2. **내 문체 가이드 만들기**: 내가 쓴 글 다섯 편 이상을 주고 「내 블로그 문체 가이드 만들어줘」라고 한다. 다음 윤문부터 선택지에 오른다.
3. **내 취향 쌓기**: 결과물을 코멘트용으로 올리고, 걸리는 문장마다 「내가 쓴다면 이렇게 쓴다」를 단 뒤 「코멘트 반영해줘」라고 한다.

Python 3과 Claude Code 또는 Codex만 있으면 된다. 별도 서버·DB·모델 설정이 없고, 검사기는 표준 라이브러리로 돈다. 다른 프로젝트에서도 부를 수 있게 진입 스킬 한 장을 두는 방법이 README에 있다.

저장소 바로 가기

저장소 github.com/byungjunjang/slop-rewrite · 공개 사이트 slop-rewrite.vercel.app(윤문 방식과 전후 사례) · MIT 라이선스 · 2026-11-08 공개본 기준 · 이 책의 윤문 기록은 발행자가 보관